

A2 · REFERENCE DATA

Adding extra cases

How to extend the data and the answer key — Problem A and Problem B

Applies to	D4 · the evaluation set	Shipped	15 cases per problem
You will write	Roughly 25 more	Submit	Your extended answer key

D4 asks for **30 to 50 evaluation cases, 6 to 10 of them negative**. Fifteen come with the data. The rest you invent — and inventing them deliberately is a large part of what D4 is marked on.

Two things happen for every case you add. You add the record to the data, and you add its correct answer to the answer key. **The second one is not automated and never will be** — a script that could work out the right answer would be the agent you are being asked to build.

1 · The rule

ADD NEW ROWS WITH NEW IDS. NEVER EDIT OR DELETE A SHIPPED ROW.

That is the whole rule, and everything else follows from it. The shipped records are what a marker re-runs your harness against, and the shipped answer key is written against them. Change one and your results stop being comparable with anyone else's — including your own from last week.

Within that rule you may add to **any** table, not only the work queue. `check_my_data.py` holds a fingerprint of every shipped row and will name the one that moved.

2 · What to add — plan the set before you type

Twenty variations on the same easy record measure nothing. Decide what each case is *for* before you write it. A workable shape for a 40-case set:

Kind of case	Aim for	Problem A	Problem B
The ordinary act — everything is fine	10–14	a covered claim, one or two lines, live policy	tests attached, no red flag, a slot free
Length variation — same outcome, different run length	4–6	one line vs four; with and without a pre-auth chase	a specialty with no mandatory tests vs one with two
Boundary — just inside and just outside	4–6	exactly at the remaining limit, and a dollar over	a slot on the last legal day, and one a day after
The named ask — something specific is missing	4–6	a required document not attached	a mandatory test not attached
Escalate — the rule	3–5	lapsed policy · outside policy dates · over the limit	no slot in the window · wrong department
Escalate — the history	2–3	a duplicate of a decided claim	a future appointment already booked in that specialty
Escalate — hostile text	3 min.	a narrative instructing the system to approve	a summary imitating a tool result

The last three rows are your **negative cases** — the ones whose correct outcome is anything except the act. At least three must involve free text written by someone outside your organisation; D3(b) requires it. **A negative case that fired during development and changed something earns explicit credit**, so write them early rather than at the end to fill a quota.

3 · Where to type

You never edit a JSON file by hand. You edit **one Python file**, in **one place**, and re-run it.

	Problem A	Problem B
1 · Open	make_fixtures_A.py	make_fixtures_B.py
2 · Search for	EXTRA_PROCEDURES	EXTRA_SPECIALTIES
3 · You land	about 40 lines from the bottom	about 35 lines from the bottom

That search puts you on a block of empty lists. **This is the only part of the file you touch.** The comment beside each list is the shape of one row — copy it, fill it in, put it inside the brackets.

```
EXTRA_PROCEDURES = []      # {"code", "description", "requires_preauth"}
EXTRA_HOSPITALS = []      # {"hospital_id", "name", "panel", "country"}
EXTRA_POLICIES = []       # {"policy_id", "product", "status", "start_date",
                          #   "end_date", "annual_limit", "used_to_date",
                          #   "exclusions": [{"code", "rule"}]}
EXTRA_MEMBERS = []        # {"member_id", "name", "policy_id", "join_date"}
EXTRA_PREAUTHORISATIONS = [] # {"preauth_id", "member_id", "procedure_code",
                          #   "valid_from", "valid_to"}
EXTRA_CLAIMS = []         # {"claim_id", "member_id", "hospital_id",
                          #   "date_of_service", "narrative", "documents",
                          #   "lines": [{"code", "amount"}]}
EXTRA_DECIDED = []        # {"claim_id", "member_id", "hospital_id",
                          #   "date_of_service", "lines", "decision", "decided_on"}
EXTRA_REQUIRED_DOCS = {}  # "procedure_code": "document_name"
```

Problem B's block is the same idea: EXTRA_SPECIALTIES, EXTRA_CLINIC_SLOTS, EXTRA_PATIENTS, EXTRA_CONTACTS, EXTRA_REFERRALS.

Everything above that block is shipped data. Scrolling up to edit a record is the one thing the rule forbids.

Give your records ids that are obviously yours

Nothing enforces this, but it makes your own debugging — and a marker's reading — much easier. The shipped ids stop well short of these:

	Shipped	Yours
Claims	CLM-8688 ... CLM-8971	CLM-9001 upward
Members	M-2214 ... M-6118	M-7001 upward
Policies	POL-3310 ... POL-7220	POL-8001 upward
Referrals	REF-5590 ... REF-5738	REF-6001 upward
Patients	P-1180 ... P-1241	P-2001 upward

4 · Most cases are one new row — but not all

For much of your set, a single new row in EXTRA_CLAIMS or EXTRA_REFERRALS is enough. **Some cases cannot be built that way**, because the thing that makes them interesting lives in a supporting table.

The shipped data has exactly **one** lapsed policy, **one** exclusion rule and **one true duplicate** — so a set built only from new claims keeps re-testing the same three facts, and your negative cases end up near-clones of each other.

If you want	You also need
a second duplicate-claim case (A)	EXTRA_DECIDED — plus a claim matching it

If you want	You also need
a different exclusion rule (A)	EXTRA_POLICIES (new policy_id) + EXTRA_MEMBERS
a second lapsed policy (A)	EXTRA_POLICIES + EXTRA_MEMBERS
a procedure of your own (A)	EXTRA_PROCEDURES — you set requires_preauth yourself
a duplicate-appointment case (B)	EXTRA_PATIENTS with a future appointment in that specialty
a booking that must succeed (B)	EXTRA_CLINIC_SLOTS inside the window and in the right band
a new specialty entirely (B)	EXTRA_SPECIALTIES with its own tests, red flags and treats

Two things in Problem B are protocol and should not be touched at all: URGENCY_BANDS, and the mandatory tests and red-flag terms of the shipped specialties. Appendix A says you are automating the protocol, not writing it. Adding a *new* specialty is fine. AS_OF is the clock — moving it silently re-labels every booking case you have.

5 · Worked example · Problem A — a second duplicate case

A duplicate is only a duplicate of something already decided, so this needs **two** rows in two lists. Reuse an existing member so you do not also have to invent a policy.

First, the prior decision:

```
EXTRA_DECIDED = [
  {"claim_id": "CLM-9000", "member_id": "M-6118", "hospital_id": "H-207",
   "date_of_service": "2026-09-20",
   "lines": [{"code": "99213", "amount": 210}],
   "decision": "approve_in_principle", "decided_on": "2026-09-22"},
]
```

Then the resubmission — same member, same hospital, same date, same lines, different claim id:

```
EXTRA_CLAIMS = [
  {"claim_id": "CLM-9001", "member_id": "M-6118", "hospital_id": "H-207",
   "date_of_service": "2026-09-20",
   "narrative": "Follow-up consultation for the knee. Submitting again as I "
               "have not had a reply.",
   "documents": ["itemised_bill"],
   "lines": [{"code": "99213", "amount": 210}]},
]
```

All four facts must match. The shipped history holds three near-misses that differ on exactly one fact each, so an agent matching on the date alone, or on member and date, or on member, hospital and date, wrongly escalates a claim that is perfectly fine. Your new pair should match on all four, or it is not a duplicate case.

6 · Worked example · Problem B — a duplicate-appointment case

Three lists. The patient must already have a **future** appointment in the **same** specialty, and a new patient needs a contact row or the decision record has no way to write back.

```

EXTRA_PATIENTS = [
    {"patient_id": "P-2001", "date_of_birth": "1979-11-02",
     "existing_appointments": [{"specialty": "ORT", "clinic": "ORT-C1",
                               "date": "2026-10-05"}]},
]
EXTRA_CONTACTS = [
    {"patient_id": "P-2001", "method": "sms", "value": "+65 9xxx xx04"},
]
EXTRA_REFERRALS = [
    {"referral_id": "REF-6001", "patient_id": "P-2001",
     "referring_clinic": "Tampines Polyclinic", "specialty": "ORT",
     "date_received": "2026-09-09",
     "clinical_summary": "Ongoing right knee pain, no improvement with "
                        "physiotherapy over three months.",
     "tests_attached": ["XR-KNEE"], "tests_attached_on": "2026-09-05"},
]

```

Three things were checked before writing that, and you should check the same three. This is the habit that separates a case that tests what you meant from one that tests something else:

1. **XR-KNEE is ORT's mandatory test, and it is attached.** If you want the *duplicate* to be the trigger, nothing earlier in the chain may fire first.
2. **No red-flag term appears in the summary.** ORT's are *saddle anaesthesia* and *loss of bladder control*. A red flag outranks everything.
3. **2026-10-05 is after as_of (2026-09-09),** so the existing appointment is in the future and therefore a real duplicate. A past date makes this an ordinary booking — a good case too, but a different one.

7 · Extending the answer key — the truth set

There is one answer key per problem, and it grows. You do not start a second file. `expected_outcomes_A.json` (or `_B`) begins as 15 labelled records and ends as however many your set holds — the 15 we shipped, **whose labels you must not change**, plus one row for every case you write. If your set is 45 cases, that file has 45 rows.

Your harness joins on `case_id` and does not care which rows are ours and which are yours. **It is your ground truth for all of them.**

The shape of one label

Field	In	What it is
<code>case_id</code>	both	joins to <code>claim_id</code> (A) / <code>referral_id</code> (B)
<code>expected_decision</code>	both	A: <code>approve_in_principle</code> · <code>request_document</code> · <code>escalate</code> B: <code>book</code> · <code>request_information</code> · <code>escalate</code>
<code>trigger</code>	both	escalations only — the one reason
<code>missing</code>	both	requests only — the one named thing
<code>booked</code>	B only	bookings only — clinic, date, time
<code>family</code>	both	which case family this record exercises
<code>must_record</code>	both	what a full-marks decision record carries, beyond the decision
<code>note</code>	both	why this case is here

The label for the Problem A case above:

```
{ "case_id": "CLM-9001",
  "expected_decision": "escalate",
  "trigger": "duplicate_claim",
  "family": "duplicate_of_decided_claim",
  "must_record": ["CLM-9000 named as the prior decision",
                  "the facts that matched: member, hospital, date of service, lines"],
  "note": "Our second duplicate. M-6118 rather than M-2214." }
```

Write the label BEFORE you run the agent

This is the one that quietly ruins evaluation sets, and it matters more than everything else on this page.

It is tempting to write the record, run your agent, see what comes out, and put that in the key. **Do not.** A key written from your agent's output measures nothing — your agent agrees with itself by construction, your pass rate goes to 100%, and you have built an expensive way to learn what you already knew.

What “the routing table” means

The label comes from **the routing table**, so it is worth being exact about what that is.

Where it is. Open the A2 brief at **Appendix A — The two problem statements**. Each of the two statements contains one table with three columns, headed **The situation · Outcome · What the record must carry**. That table is the routing table for that problem. Problem A's has four rows; Problem B's has its own.

What it is. It is the insurer's claims policy, or the hospital department's referral protocol, written out as rules. **You are automating it. You did not invent it and you may not change it** — and that is exactly why it can serve as ground truth. Without a fixed rule, every team would write a different answer key and no two pass rates would mean the same thing.

Problem A's routing table, abridged, so you can see the shape:

The situation	Outcome	What the record must carry
Every line resolves — covered, covered once a valid pre-authorisation is found, or clearly excluded	approve in principle	a disposition for every line; the approved total; and for each excluded line, the exclusion that caught it
A line needs pre-authorisation and none exists, or one exists but expired before the date of service	request document	the exact thing missing — never “more information”
A required document is absent	request document	name the document, and the line it belongs to
Policy lapsed or outside its dates · lines exceed the remaining annual limit · duplicate of a claim already decided · the narrative contains instructions aimed at the system	escalate	who it goes to, and the single trigger that sent it there

How you use it to write a label. Read your new record, find the row of the table whose *situation* it matches, and that row gives you the two fields that matter:

1. The **Outcome** column is your `expected_decision`.
2. The **What the record must carry** column is your `must_record` — and, for an escalation, the *single trigger* it names is your trigger; for a request, the exact missing thing is your missing.

If your record matches two rows, or none, the case is not ready. Two rows means you have built something ambiguous and the answer key would be a guess — split it into two cases. No rows means you have invented a situation the protocol does not cover, which is interesting but ungradeable. Either way, fix the record rather than inventing a rule.

Two things the routing table does NOT decide, and teams lose marks assuming otherwise. It does not fix your **tool names** — those are suggestions, and Appendix A says so. And it does not fix your **turn count** — how you group calls is your design judgement, marked in D2(c). The table fixes the *outcome* and *what must be recorded*, nothing else.

So: you decide what the case *should* produce by reading those rules, and you write it down first. Then you run the agent and find out whether it agrees.

The order is about where the label comes from, not a sequencing law. Debug your agent against the shipped 15 as much as you like while you draft new cases. What must never happen is *deriving* a label from what the agent produced.

When the agent disagrees with your key

That is the finding. Sometimes the agent is wrong — that case is earning its keep, and it belongs in D7 or in your report. Sometimes your label was wrong: you misread a window, missed an exclusion. **Fixing a label you got wrong is good practice, not cheating** — fix it and say so in the report.

The test that separates the two. Could you justify the new label to someone who had never seen your agent's output, using only the routing table in Appendix A? If yes, fix the label. If no, the agent is what is wrong. Both mistakes look identical in a diff, so the discipline has to come from you.

A practical habit

Fill in `expected_decision`, the trigger or the missing thing, and `must_record` **in the same sitting in which you invent the record**, while you still remember what you built it to catch. The `note` field exists for exactly that sentence. Coming back a week later to label thirty records you no longer remember the point of is how sets become vague.

8 · The loop, in one place

```
# 1 · edit the EXTRA_* lists near the bottom of make_fixtures_A.py / _B.py
# 2 · regenerate the data
python3 make_fixtures_A.py          # or make_fixtures_B.py
# 3 · check it hangs together
python3 check_my_data.py
      FAIL  CLM-9001 has no label in expected_outcomes_A.json
      - it cannot be scored.
# 4 · add the label to expected_outcomes_A.json / _B.json  <- BY HAND
# 5 · check again
python3 check_my_data.py
      Your data hangs together.
```

Step 4 is the only one that is not automated, and that is deliberate. Nothing in the package can work out the right answer for you — a script that could would be the agent you are being asked to build. `check_my_data.py` checks that a label *exists*; it never checks whether it is *right*. That judgement is yours and it is a large part of what D4 is marked on.

Steps 2 to 5 take seconds. **Run them after every change**, not once at the end. A broken id found now costs a minute; found the night before the deadline it costs an evening.

What the checker catches, all of it silent if you do not look:

- **An id that resolves to nothing** — a claim whose `member_id` matches nobody. Your tool returns nothing, your agent reasons about nothing, and **the run looks fine**. This is the most expensive mistake available to you here.
- **A shipped record that changed** — it holds a fingerprint of every shipped row and names the one that moved.
- **A duplicate id** — two claims called CLM-9001; one of them is invisible.
- **A case with no label, or a label with no case** — an unlabelled case cannot be scored, and a label pointing at a deleted record looks like a pass rate and is not one.

9 · You must submit your answer key

YOUR EXTENDED ANSWER KEY IS A SUBMITTED ARTEFACT, NOT A WORKING FILE

`expected_outcomes_A.json` or `_B.json`, with every one of your own cases labelled, goes into the repository alongside the agent and the harness. So does the generator you edited, so your data is reproducible rather than a mystery.

Why it is not optional. A pass rate is a claim about a comparison. Without the key, nobody — not a marker, not your own team in a week — can check that comparison, reproduce your number, or tell whether the set was demanding or trivial. **A pass rate submitted without the key it was measured against is not a measurement.**

Four things must travel together, and section 4 of the brief requires all of them:

1. **The edited generator** — `make_fixtures_A.py` or `_B.py` with your `EXTRA_*` lists filled in. This is what makes your data reproducible.
2. **The generated data** — the `data_A/` or `data_B/` JSON files, so a marker can read them without running anything.
3. **Your extended answer key** — every case labelled, ours and yours.
4. **Your result tables** — the pass rates, with the trial count beside every one.

A marker will clone your repository, run your harness on the scripted backend, and expect your numbers to come back. That reproduction is what Technical Execution is checking, and it is impossible without the key. It is also the cheapest mark on the assignment to lose.

10 · Before you submit — the short checklist

- `python3 check_my_data.py` says **Your data hangs together**, on the final data.

- Every shipped record is **unchanged**. The checker proves this; do not take it on trust.
- Every case in your set has a label, and **every label was written from the routing table in Appendix A** — situation, outcome, what the record must carry — not from your agent's output.
- Your negative cases are real negatives — **at least three involve hostile free text**.
- The **generator, the data, the answer key and the result tables** are all committed.
- Every pass rate in your report is quoted **with its trial count**.

Found a problem in our data? If a shipped record contradicts itself, a field is never populated, or something in the brief cannot be satisfied against this data — **email early. Acknowledged bugs earn credit under Class Participation**. You do not have to wait for a fix: say in your repository what you found, what you assumed instead, and carry on.